

Setting up NGINX for PHP

From Development Server to Production-Ready
Web Server

What We're Building

Current Setup (Development)

```
php -S localhost:8000
```

- Single-threaded
- Development only
- No static file optimization
- Limited concurrent users

New Setup (Production-Ready)

NGINX + PHP-FPM (PHP FastCGI Process Manager)

- Event-driven NGINX + PHP-FPM worker processes
- Production ready
- Optimized static file serving
- Handles many more concurrent users

Understanding the Architecture

| PHP Built-in Server

Browser → PHP -S → PHP Script

Single process handles everything

NGINX + PHP-FPM

Browser → NGINX → PHP-FPM → PHP Script
↓
Static Files (direct)

**NGINX handles static files, PHP-FPM processes
PHP**

NGINX and PHP setup is different on macOS and Linux in this tutorial, so the configuration files and commands are not the same.

Read the instructions for your operating system carefully before editing any config files.

If you run into problems, check the platform-specific section below before making changes.

Step 1: Install PHP-FPM

macOS

```
# PHP-FPM comes with Homebrew PHP  
brew install php
```

```
# Or if you need to reinstall  
brew reinstall php
```

```
> brew list php | grep php-fpm  
/opt/homebrew/Cellar/php/8.4.11/sbin/php-fpm  
/opt/homebrew/Cellar/php/8.4.11/.bottle/etc/php/8.4/php-fpm.d/www.conf  
/opt/homebrew/Cellar/php/8.4.11/.bottle/etc/php/8.4/php-fpm.conf  
/opt/homebrew/Cellar/php/8.4.11/.bottle/var/log/php-fpm.log  
/opt/homebrew/Cellar/php/8.4.11/share/man/man8/php-fpm.8
```

Linux (WSL2/Ubuntu)

```
sudo apt update  
sudo apt install php-fpm php-mysql
```

You need to check the version number, in this example, it is 8.3.

```
Processing triggers for man-db (2.12.0-4build2) ...  
Processing triggers for php8.3-cli (8.3.6-0ubuntu0.24.04.10) ...  
Processing triggers for php8.3-fpm (8.3.6-0ubuntu0.24.04.10) ...
```

Step 2: Start PHP-FPM Service

macOS

```
brew services start php  
# or manually: php-fpm
```

`brew services` command shows the services that are running on your system.

```
smcho@m4 www> brew services  
Name  Status  User  File  
mysql started smcho ~/Library/LaunchAgents/homebrew.mxcl.mysql.plist  
nginx started smcho ~/Library/LaunchAgents/homebrew.mxcl.nginx.plist  
php   started smcho ~/Library/LaunchAgents/homebrew.mxcl.php.plist
```

Linux

Start the php-fpm, change 8.3 into the version for your system if it is not 8.3.

```
sudo systemctl start php8.3-fpm      # Ubuntu (adjust version)
sudo systemctl enable php8.3-fpm     # Auto-start on boot
```

Step 3: Basic NGINX Configuration for PHP

macOS

Find Your NGINX Configuration File

- `/usr/local/etc/nginx/nginx.conf` or `/opt/homebrew/etc/nginx/nginx.conf` for

Apple Silicon Mac

Create a Basic Configuration: You can copy and paste the code block from (module1/code/5_Webserver_using_NGINX).

```
server {
    listen 8080;
    server_name localhost;
    root /opt/homebrew/var/www; # Adjust path for your system
    index index.php index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        try_files $uri =404;
        fastcgi_pass 127.0.0.1:9000; # PHP-FPM address
        fastcgi_index index.php;
        fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;
        include fastcgi_params;
    }
}
```

macOS uses port 9000 for the communication between PHP and NGINX, so use `lsof` command to check if the port 9000 is working correctly.

```
└─> lsof -nP -iTCP:9000 -sTCP:LISTEN
COMMAND  PID  USER  FD  TYPE  DEVICE  SIZE/OFF  NODE  NAME
php-fpm  69823  chos5   9u  IPv4  0x316a12bb83c0e785  0t0  TCP  127.0.0.1:9000 (LISTEN)
php-fpm  69824  chos5  10u  IPv4  0x316a12bb83c0e785  0t0  TCP  127.0.0.1:9000 (LISTEN)
php-fpm  69825  chos5  10u  IPv4  0x316a12bb83c0e785  0t0  TCP  127.0.0.1:9000 (LISTEN)
```

1. Replace only the server section, not the whole section!
 - Use `sudo nginx -t` command to check.
2. Restart the server after updating the nginx.conf.
3. Make sure the PHP code is located in the root directory.

| Linux

Find Your NGINX Configuration File

- **Linux:** `/etc/nginx/nginx.conf`

Create a Basic Configuration: You can copy and paste the code block from (module1/code/5_Webserver_using_NGINX).

```
# Careful! Replace the server section, not the whole section!
server {
    listen 80;
    server_name localhost;
    root /var/www/html; # Adjust path for your system
    index index.php index.html index.htm;

    location / {
        try_files $uri $uri/ =404;
    }

    location ~ \.php$ {
        try_files $uri =404;
        include snippets/fastcgi-php.conf;
        fastcgi_pass unix:/run/php/php8.3-fpm.sock; # Change 8.3 to your PHP version if different
    }
}
```

Linux uses socket for the communication between PHP and NGINX, so you need to check if the socket is available.

```
└─> ls -al /run/php/
total 4
drwxr-xr-x  2 www-data www-data 100 Sep  8 10:46 .
drwxr-xr-x 21 root      root    700 Sep  8 10:45 ..
lrwxrwxrwx  1 root      root     30 Sep  8 08:59 php-fpm.sock -> /etc/alternatives/php-fpm.sock
-rw-r--r--  1 root      root      5 Sep  8 10:46 php8.3-fpm.pid
srw-rw----  1 www-data www-data   0 Sep  8 10:46 php8.3-fpm.sock
```

Linux NGINX configuration includes files in the `/etc/nginx/sites-enabled` automatically.

- Copy the server configuration in the directory.
- Make a symbolic link in the `/etc/nginx/sites-enabled` so the configuration is read and uploaded.
- Follow the instructions in the next section.

1. Replace `/etc/nginx/sites-available/default` with this file! Be sure to copy the original default file
 - Use `sudo nginx -t` command to check.
2. Restart the server after updating the nginx.conf.
3. Make sure the PHP code is located in the root directory.

1. On Ubuntu/Debian, `sites-available` contains configured site files, while `sites-enabled` contains the active ones. A config is "enabled" when a symlink is created in `sites-enabled` pointing to a file in `sites-available`.
- `sites-available` = saved configuration candidates
- `sites-enabled` = active configurations actually used by nginx

Example commands:

In this example, we copy the `nginx.conf` into `myapp` to store in the `sites-available`, and make a symbolic link in the `sites-enabled`.

```
sudo cp nginx.conf /etc/nginx/sites-available/myapp
sudo ln -s /etc/nginx/sites-available/myapp /etc/nginx/sites-enabled/myapp
sudo nginx -t
sudo systemctl reload nginx
```

Server Block Overview

Make sure you have the correct **root** for your system!

```
server {  
    listen 8080;  
    server_name localhost;  
    root /opt/homebrew/var/www; # <--  
    index index.php index.html;  
}
```

You need to understand the settings:

- **listen 8080**: HTTP port (use 8080 on macOS instead of 80)
- **server_name localhost**: Domain (change to your own domain if needed)
- **root**: Folder to serve files (adjust path; use  not  on Windows)
- **index**: Default file(s) to load in a directory

Static File Handling

```
location / {  
    try_files $uri $uri/ =404;  
}
```

- Tries to serve the exact file or folder
- If not found, returns 404
- Note: this does *not* automatically map `/about` to `/about.php`; PHP routing must still be handled explicitly

PHP File Handling

This is the connection between NGINX and PHP (macOS example).

```
location ~ /\.php$ {  
    try_files $uri =404;  
    fastcgi_pass 127.0.0.1:9000;  
    fastcgi_index index.php;  
    fastcgi_param SCRIPT_FILENAME $document_root$fastcgi_script_name;  
    include fastcgi_params;  
}
```

- `try_files $uri =404`: returns 404 if the PHP file doesn't exist, instead of forwarding it to PHP-FPM
- Matches .php files via regex
- Forwards request to PHP-FPM at 127.0.0.1:9000
- Passes script path and required FastCGI parameters

Linux uses socket for the connection.

```
location ~ /\.php$ {  
    try_files $uri =404;  
    include snippets/fastcgi-php.conf;
```

How It Works

1. Browser requests

<http://localhost:8080/index.php>

2. NGINX matches .php and forwards to PHP-FPM

3. PHP-FPM executes index.php and returns output

4. NGINX sends a response back to the browser

Efficient, production-ready setup for PHP apps

| Step 4: Restart nginx

- **Mac:** `nginx -s reload`
- **Linux:** `sudo systemctl restart nginx`

Step 5: Create a Test PHP File

Create `info.php` in your web root

```
<?php  
phpinfo();  
?>
```

Security note: `phpinfo()` exposes detailed server and PHP configuration. Delete `info.php` from your web root once you're done testing.

Create **php_fpm_test.php** for API testing

```
<?php
header('Content-Type: application/json');

$response = [
    'message' => 'NGINX + PHP-FPM is working!',
    'server' => $_SERVER['SERVER_SOFTWARE'] ?? 'Unknown',
    'php_version' => PHP_VERSION,
    'timestamp' => date('Y-m-d H:i:s'),
    'method' => $_SERVER['REQUEST_METHOD'],
    'uri' => $_SERVER['REQUEST_URI']
];

echo json_encode($response, JSON_PRETTY_PRINT);
?>
```

- PHP automatically generates this as a response.

```
HTTP/1.1 200 OK
Server: nginx/1.24.0
Content-Type: application/json
Content-Length: 226
Connection: keep-alive

{
  "message": "NGINX + PHP-FPM is working!",
  "server": "nginx/1.24.0",
  "php_version": "8.2.12",
  "timestamp": "2025-08-02 18:01:30",
  "method": "GET",
  "uri": "/php_fpm_test.php"
}
```

Step 6: Start Services and Test

1. Start PHP-FPM (if not already running)

macOS:

```
brew services start php  
# php-fpm for manual start
```

Linux:

```
sudo systemctl start php8.3-fpm # Ubuntu
```

| 2. Reload NGINX Configuration

macOS & Linux:

```
nginx -s reload
```

| 3. Test Your Setup

- Visit: <http://localhost/info.php> (macOS: <http://localhost:8080/info.php>)
- Visit: http://localhost/php_fpm_test.php (macOS: http://localhost:8080/php_fpm_test.php)

You should see the PHP info and the JSON response!